

Aula 000 - Exercícios Resolvidos

Soluções comentadas - Introdução aos Paradigmas de Programação

Como usar esta versão

As respostas abaixo são modelos. Questões argumentativas podem aceitar outras respostas, desde que sejam tecnicamente coerentes e bem justificadas.

Parte A - Conceitos fundamentais

1. Por que estudar paradigmas?

Três benefícios possíveis: (1) ampliar a capacidade de expressar soluções, pois o desenvolvedor passa a conhecer diferentes estruturas e abstrações; (2) escolher linguagens e recursos de forma mais fundamentada, em vez de usar apenas a ferramenta familiar; (3) aprender novas linguagens com mais rapidez, reconhecendo conceitos já conhecidos sob outra sintaxe. Também são válidos: compreender melhor a implementação, usar melhor linguagens já conhecidas e avaliar criticamente decisões de projeto.

2. Linguagem, paradigma e domínio

Linguagem de programação é um sistema formal para descrever computações. Paradigma é uma forma predominante de pensar e estruturar a solução. Domínio de aplicação é a classe de problemas em que o software será usado. Exemplo: Python é uma linguagem multiparadigma; em análise de dados, pode ser usada com estilo imperativo ou funcional para transformar coleções.

3. Sintaxe ou semântica?

a) Sintaxe. b) Semântica. c) Sintaxe. d) Semântica. e) Normalmente sintaxe ou erro estático de chamada, dependendo da linguagem. f) Semântica. A distinção central é: sintaxe trata da forma válida; semântica trata do significado e do comportamento.

4. Etapas de tradução

Uma sequência simplificada é: análise léxica -> análise sintática -> análise semântica -> geração de código -> execução. A análise léxica cria tokens; a sintática verifica a estrutura e produz uma representação hierárquica; a semântica verifica significado estático, tipos e coerência; a geração de código produz uma forma executável ou intermediária; a execução realiza as instruções no ambiente alvo.

Parte B - Paradigmas em ação

5. Identificação de paradigmas

a) Imperativo. b) Orientado a objetos. c) Funcional. d) Lógico. e) Concorrente. Na prática, linguagens e programas podem combinar esses estilos.

6. Curto-circuito

a) Quando x é zero, a primeira parte é falsa e a segunda não é avaliada. b) Sem curto-circuito, ocorreria tentativa de dividir por zero. c) Uma forma explícita é:

```
se x != 0:
    se 10 / x > 2:
        mostrar("condição verdadeira")
    senão:
        mostrar("condição falsa")
senão:
    mostrar("condição falsa")
```

7. Mesmo problema, estilos diferentes

A Versão A é mais imperativa porque usa variável acumuladora, laço e alteração explícita de estado. A Versão B usa funções de ordem superior e composição, aproximando-se de um estilo funcional. Para uma equipe iniciante, a Versão A pode ser mais fácil de acompanhar passo a passo; uma alternativa funcional mais legível em Python seria usar uma expressão geradora:

```
total = sum(numero for numero in numeros if numero % 2 == 0)
```

Parte C - Escolhas e análise crítica

8. Escopo e tempo de vida

x foi definido no escopo externo e pode ser lido dentro da função. y existe no escopo da função. z existe apenas dentro do bloco da condição, em linguagens que adotam escopo de bloco. Assim, mostrar(z) fora do bloco pode ser inválido ou, em algumas linguagens, depender de a condição ter sido executada. A resposta correta deve mencionar que regras exatas variam entre linguagens.

9. Escolha de linguagem

Python é uma escolha plausível porque a equipe já domina a linguagem, o prazo é curto, existe amplo ecossistema para análise de dados e o desempenho exigido é moderado. Critérios relevantes: produtividade, experiência da equipe, bibliotecas, legibilidade, manutenção, integração, desempenho real necessário, disponibilidade de profissionais e custo operacional. A linguagem de baixo nível só seria preferível se houvesse requisito mensurável que justificasse o custo adicional de desenvolvimento e manutenção.

10. Trade-offs

Não existe uma linguagem melhor em tudo porque critérios entram em conflito. Exemplo 1: maior controle manual de memória pode melhorar desempenho e previsibilidade, mas aumenta o risco de erros. Exemplo 2: uma sintaxe muito concisa reduz quantidade de código, porém pode prejudicar a legibilidade. Outro exemplo: verificações de segurança em tempo de execução aumentam confiabilidade, mas consomem recursos.

Parte D - Aplicação e síntese

11. Linguagem multiparadigma

Exemplo com Python: no estilo imperativo, usamos laços e atribuições; no funcional, usamos map, filter, funções lambda e compreensões; no orientado a objetos, definimos classes e objetos. Um mesmo programa pode misturar estilos, mas a mistura deve ter propósito e preservar a legibilidade.

```
# Imperativo
dobrados = []
for n in numeros:
    dobrados.append(n * 2)

# Funcional
dobrados = list(map(lambda n: n * 2, numeros))

# Orientado a objetos
class Dobrador:
    def aplicar(self, n):
        return n * 2
```

12. Inteligência artificial e responsabilidade

Verificações esperadas: compreender o código; testar casos normais e extremos; confirmar requisitos; avaliar segurança; revisar dependências; verificar tratamento de erros; medir desempenho; conferir licenças; remover dados sensíveis; executar análise estática; revisar estilo e manutenção; validar com outro desenvolvedor. Compilar não prova que a solução está correta.

13. Desafio prático

Uma solução imperativa possível em Python:

```
numeros = [4, -2, 7, 8, 3]

soma_pares = 0
positivos = 0
maior = numeros[0]

for numero in numeros:
    if numero % 2 == 0:
        soma_pares += numero
    if numero > 0:
        positivos += 1
    if numero > maior:
        maior = numero

print(soma_pares, positivos, maior)
```

O paradigma predominante é imperativo. Uma reescrita com recursos funcionais poderia usar sum, filter e max:

```
soma_pares = sum(filter(lambda n: n % 2 == 0, numeros))
positivos = sum(1 for n in numeros if n > 0)
maior = max(numeros)
```

Síntese possível

Estudar paradigmas amplia as formas de pensar sobre um problema, melhora a escolha das ferramentas e permite construir soluções mais claras, adequadas e responsáveis.

Observação pedagógica: a lista foi produzida como material autoral da disciplina. Ela trabalha conceitos presentes na aula introdutória sem reproduzir exercícios de obras de referência.